```
==  CreateProducts: migrated (0.0023s) ======================================
```

And that's it. Rails looks for all the migrations not yet applied to the database and applies them. In our case, the `products` table is added to the database defined by the `development` section of the `database.yml` file.

OK, all the groundwork has been done. We set up our Depot application as a Rails project. We created the development database and configured our application to be able to connect to it. We created a products controller and a `Product` model and used a migration to create the corresponding `products` table. And a number of views have been created for us. It's time to see all this in action.

Seeing the List of Products

With three commands, we've created an application and a database (or a table inside an existing database if you chose something besides SQLite 3). Before we worry too much about what happened behind the scenes here, let's try our shiny new application.

We mentioned previously that using a CSS processor will affect how we start our server during development. This is because things like CSS processors and JavaScript bundlers require a build step. Rather than requiring you to start multiple processes, Rails provides `bin/dev`, which is a small script that will start everything:

```
depot> bin/dev
10:17:38 web.1  | started with pid 75099
10:17:38 css.1  | started with pid 75100
10:17:39 web.1  | => Booting Puma
10:17:39 web.1  | => Rails 7.0.4 application starting in development
10:17:39 web.1  | => Run `bin/rails server --help` for more startup options
10:17:39 web.1  | Puma starting in single mode...
10:17:39 web.1  | * Puma version: 5.5.2 (ruby 3.1.1p18) ("Zawgyi")
10:17:39 web.1  | *  Min threads: 5
10:17:39 web.1  | *  Max threads: 5
10:17:39 web.1  | *  Environment: development
10:17:39 web.1  | *          PID: 75099
10:17:39 web.1  | * Listening on http://127.0.0.1:3000
```